authcore 插件深度审查报告

> 版本：0.1.22 | 类型：后端插件 | 代码量：~2,200 行
>
> 审查日期：2026-03-27

综合评分

维度	评分	简评
功能性	8/10	功能全面，支持多种认证方式
安全性	2/10	多个致命安全漏洞
架构设计	7/10	分层清晰，但 InMemory 实现流入生产
接口设计	6/10	抽象合理但命名不一致
错误处理	3/10	大量静默失败
可测试性	5/10	DI 设计良好但无实际测试
总评	**4.5/10**	架构骨架扎实，安全实现灾难级

一、功能性分析

支持的认证方式（✅ 全面）

1. **邮箱 + 密码**：注册、登录、重置密码
2. **手机 + 密码**：注册、登录
3. **邮箱验证码**：发送 + 验证
4. **短信验证码**：腾讯云 SMS 集成
5. **Google OAuth**：IdToken 验证
6. **Apple OAuth**：IdentityToken 验证（JWKS）
7. **微信 OAuth**：UnionId 绑定
8. **游客登录**：匿名创建
9. **双协议支持**：REST JSON + Protobuf 二进制

功能缺陷

缺陷	严重度	说明
无账号锁定机制	🔴 严重	登录失败无次数限制，可暴力破解
无多设备管理	🟠 高	登录即踢掉所有旧会话，无选择性
无密码强度策略	🟠 高	任意密码均可注册
手机号仅支持中国	🟡 中	硬编码 `^1[3-9]\d{9}$` 正则
无审计日志	🟡 中	登录/登出无追踪记录

二、安全性批判

🔴 致命问题 1：InMemoryPasswordHasher 是笑话级实现

```
class InMemoryPasswordHasher implements PasswordHasher {
  @override
  Future<String> hash(String password) async => 'hash:${password.length}';
  @override
  Future<bool> verify(String password, String hash) async => hash == await
this.hash(password);
}
```

批判：

- 仅用密码长度做 "哈希"，密码 `12345678` 和 `abcdefgh` 哈希值完全相同
- 一个 TODO 注释就想挡住生产环境使用？这是妄想
- 没有任何运行时断言阻止它在生产环境被实例化
- 这不是哈希，这是自欺欺人

改良方案：

```
class BcryptPasswordHasher implements PasswordHasher {
  @override
  Future<String> hash(String password) async {
    assert(password.length >= 8, '密码长度不足');
    return DBCrypt().hashpw(password, DBCrypt().gensalt(rounds: 12));
  }
}

// 同时删除 InMemoryPasswordHasher，或在构造函数中加入断言：
InMemoryPasswordHasher() {
  assert(() {
    throw StateError('InMemoryPasswordHasher 禁止在生产环境使用');
```

```
  }());
}
```

🔴 致命问题 2：JWT 安全验证全部关闭

```
static Map<String, dynamic> parseToken(String token, String tokenSecret) {
  final jwt = JWT.tryVerify(token, SecretKey(tokenSecret),
      checkExpiresIn: false,     // 过期令牌照用
      checkHeaderType: false,    // 篡改 header 照过
      checkNotBefore: false);    // 未生效令牌照用
  if (jwt == null) return {};    // 验证失败？返回空 map，静默放行
  return jwt.payload;
}
```

批判：

- 关闭过期检查 = 令牌永不失效 = 一旦泄露，永久有效
- 返回空 `{}` 而非抛异常 = 调用者无法区分"未认证"和"解析失败"
- 这三个 `false` 把 JWT 规范的安全保障全部废掉了
- 相当于门锁装了但永远不锁

改良方案：

```
static Map<String, dynamic> parseToken(String token, String tokenSecret) {
  try {
    final jwt = JWT.verify(token, SecretKey(tokenSecret),
        checkExpiresIn: true,
        checkHeaderType: true,
        checkNotBefore: true);
    return jwt.payload;
  } on JWTExpiredException {
    throw AuthTokenExpiredException();
  } on JWTException catch (e) {
    throw AuthTokenInvalidException(e.message);
  }
}
```

🔴 致命问题 3：验证码生成完全可预测

```
String _generateDigitsCode(int length) {
  const chars = '0123456789';
  final buf = StringBuffer();
  for (var i = 0; i < length; i++) {
    buf.write(chars[(DateTime.now().microsecondsSinceEpoch + i) %
chars.length]);
  }
  return buf.toString();
}
```

批判：

- 用当前时间微秒 + 偏移量生成"随机"数字？这不是随机，这是时钟
- 攻击者只需知道大致时间戳（误差 ±1 秒），即可穷举 100 万种可能，10 秒内破解
- 6 位验证码 + 无限重试次数 = 形同虚设

改良方案：

```
String _generateDigitsCode(int length) {
  final rng = Random.secure(); // 密码学安全随机数
  final buf = StringBuffer();
  for (var i = 0; i < length; i++) {
    buf.write(rng.nextInt(10));
  }
  return buf.toString();
}
```

🔴 致命问题 4：验证码无重试限制

```
bool verifyAndConsumeEmailCode(String email, String code) {
  final entry = _emailCodes[emailTrim];
  if (entry == null) return false;
  if (now.isAfter(entry.expiresAt)) {
    _emailCodes.remove(emailTrim);
    return false;
  }
  if (entry.code != code.trim()) return false; // 错了？再试！无限次！
  _emailCodes.remove(emailTrim);
  return true;
}
```

批判：

- 6 位数字验证码 = 100 万种组合
- 无速率限制，攻击者可每秒发起数千次验证
- 5 分钟有效期内，10 万次请求即可破解
- 验证码机制沦为装饰品

改良方案：

```
bool verifyAndConsumeEmailCode(String email, String code) {
  final entry = _emailCodes[emailTrim];
  if (entry == null) return false;

  // 最多允许 5 次尝试
  entry.attempts++;
  if (entry.attempts > 5) {
    _emailCodes.remove(emailTrim);
    return false; // 超过次数，销毁验证码
  }

  if (now.isAfter(entry.expiresAt)) {
```

```
    _emailCodes.remove(emailTrim);
    return false;
  }
  if (entry.code != code.trim()) return false;
  _emailCodes.remove(emailTrim);
  return true;
}
```

🔴 致命问题 5：InMemoryTokenService 的刷新令牌

```
final refresh = 'r_${userId}_${_rng.nextInt(1 << 30)}';
```

- `Random()` 非密码学安全，可预测
- 30 位随机数空间仅 ~10 亿，暴力破解可行
- 服务器重启 = 所有用户强制登出
- 无令牌吊销能力

三、架构设计批判

优点

✅ **分层清晰**：接口层 → 实现层 → 路由层 → 服务层，依赖关系合理

✅ **DI 设计**：通过接口注入 `UserRepository`、`PasswordHasher`、`TokenService`

✅ **双协议设计**：REST 和 Protobuf 共享同一 `AuthService`，避免逻辑分叉

✅ **外部身份映射**：OAuth 提供者到内部用户 ID 的映射设计合理

缺陷

问题 1：*InMemory* 实现不应存在于库中

```
auth_core/
├── interfaces.dart       ← 干净的抽象
├── auth_service_impl.dart ← 正确的实现
└── in_memory_impl.dart    ← 💣 定时炸弹
```

InMemory 实现应该放在 `test/` 目录或单独的 `auth_core_test_utils` 包中，而非与生产代码一起导出。当前设计让调用者极易"图方便"在生产代码中使用。

问题 2：*CodeService* 状态不可外部化

`_emailCodes` 和 `_smsCodes` 是内存 Map，无法：

- 跨进程共享（水平扩展时验证码失效）
- 持久化（服务重启时未验证的码全部丢失）

- 监控（无法查看待验证码数量）

应抽象为 `CodeStore` 接口，支持 Redis/MySQL 后端。

问题 3：*OAuthService* 单例 + 缓存无 *TTL*

```
static OAuthService get instance => _instance ??= OAuthService._();
```

Google/Apple 的 JWKS 公钥缓存永不过期。当 OAuth 提供者轮换密钥时，验证会一直失败直到重启服务。

四、接口设计批判

问题 1：命名不一致

```
// interfaces.dart
Future<String> issueAccessToken({
  required int userId,
  required Map<String, Object?> claims,
  DateTime? exptime  // ← "exptime" 缩写不标准
});

// 同文件其他地方
DateTime? expiresAt   // ← 标准命名
```

`exptime` vs `expiresAt`，同一文件两种风格。

问题 2：Tokens 模型职责不清

```
class Tokens {
  String accessToken;
  String refreshToken;
  DateTime? accessExpTime;
  DateTime? refreshExpTime;

  Map<String, dynamic> toJson() => {
    'accessToken': accessToken,
    'refreshToken': refreshToken,
    'accessExpTime': accessExpTime?.toIso8601String(),
    'refreshExpTime': refreshExpTime?.toIso8601String(),
  };
}
```

- 过期时间为何可空？令牌怎么可能没有过期时间？
- `toJson()` 把 refresh token 也序列化返回给前端，但 refresh token 常规做法是通过 HttpOnly cookie 返回

问题 3：错误类型全用 `Exception`

```
throw Exception('Email already registered');
throw Exception('Invalid credentials');
```

```
throw Exception('User not found');
```

全部使用通用 `Exception`，调用者只能用字符串匹配来区分错误类型。应定义类型异常层级：

```
abstract class AuthException implements Exception { ... }
class EmailAlreadyRegisteredException extends AuthException { ... }
class InvalidCredentialsException extends AuthException { ... }
class UserNotFoundException extends AuthException { ... }
```

问题 4：parseToken 返回 Map 而非强类型

```
static Map<String, dynamic> parseToken(String token, String tokenSecret)
```

返回 `Map<String, dynamic>` 意味着调用者需要到处做类型转换和空值检查。应返回强类型 `TokenClaims` 对象。

五、改良建议总结

优先级	改良项	预估工作量
P0	删除 InMemoryPasswordHasher，引入 bcrypt	2h
P0	启用 JWT 全部安全检查	0.5h
P0	用 `Random.secure()` 替换验证码生成	0.5h
P0	验证码加入重试次数限制	1h
P1	定义 AuthException 层级替换通用 Exception	3h
P1	抽象 CodeStore 接口支持 Redis	4h
P1	OAuthService JWKS 缓存加 TTL	2h
P2	统一命名规范	1h
P2	Tokens 模型重构（过期时间必填 + 强类型）	2h
P2	添加密码强度策略	2h
P3	增加登录审计日志	3h
P3	支持国际手机号格式	2h